Before Generating Code

1. Read `.context/ai-rules.md` first - hard constraints that override everything
2. Read `.context/project-context.md` for project specific context.

##General Rules for code generation

1.Accuracy is much more important than speed. Take your time to work through problems and provide the best response.
2.Simplicity is preferred. Keep code as simple and straight forward as possible.
3. Solve the problem asked, nothing else. Don't add extra features/functions, don't add extra fluff unless asked for.
4. For larger tasks, automatically review and rewrite your code before presenting it to me. Do this revision until you no longer find errors or ways to make improvements. Improvements mean making the code simpler, more streamlined and straightforward, not more complicated or fancy.
5. Speed of calculations and runtime are important, once accuracy has been optimized try to make any code run as quick as possible.

##Rules for interactions
1.Don't add any extra fluff or necessities to your vocabulary. Give me the most straightforward and direct answers as possible, I don't care if you're rude or blunt, you can't hurt my feelings.
2. Ask if I would like any suggestions or recommendations in terms of next steps before providing any, do not automatically give me info that has not been asked for.
3. Provide a brief explanation of any revisions that you did that make code more complicated, and why you had to do so. If your revisions or changes made things simpler then I will ask for any explanations if I need them.
4.

Code Style Rules

Naming Conventions

```yaml
variables: camelCase
functions: camelCase
classes: PascalCase
constants: SCREAMING_SNAKE_CASE
files: kebab-case
database_tables: snake_case
database_columns: snake_case
```

Language-Specific Rules

TypeScript:
- Never use `any` - use `unknown` and narrow, or define proper types
- Always use strict mode
- Prefer `interface` over `type` for object shapes
- Use `const` by default, `let` when reassignment needed, never `var`

Go:
- Always handle errors explicitly, never ignore with `_`
- Use meaningful variable names, not single letters (except loop counters)
- Return early to avoid deep nesting

Documentation Rules

- Public functions must have doc comments
- Complex logic requires inline comments explaining "why"

Dependency Rules

- Prefer standard library over external packages
- Evaluate security and maintenance status before adding dependencies
- Pin dependency versions exactly
- Update dependencies monthly with testing

Note: These rules override any conflicting patterns. When in doubt, follow these constraints.